

A Broader Impacts and Limitations

Broader Impacts As more LMs are developed, a key criteria for their adoption and utility is if they exhibit a wide array of useful capabilities, such as generating harmless content, summarizing essays, and being conversational with the user. While improvements in other parts of the LM development pipeline such as training and architecture are important, many recent advances in building LMs with a wide array of useful capabilities have come from the data itself [9, 15, 17, 42, 56]. Our work is fundamental in investigating how LMs learn and how to select data to learn skills more efficiently. However, we recognize that data selection methods can always be utilized to optimize for particular skills that may be considered malicious or negatively target or exclude specific groups [2]. Furthermore, pre-trained LMs have been found to have various biases [6, 25, 35, 43].

Limitations The skills graph can either be provided (e.g., using a knowledge graph) or learned. Our work learns the skills graph using Algorithm 2 or Algorithm 3, which requires initial training runs on pairs of skills or each skill, respectively. This can be made more efficient by performing these training runs on a smaller model and for fewer number of steps, but tradeoffs here have yet to be thoroughly investigated. SKILL-IT also assumes that the ordered skill set is provided; as discussed in sections 2.1 and 2.3, it is challenging to recover ordered skill sets simply via metadata attributes or embedding clustering. Otherwise, the best way to sample over collections of skills that form a complete or empty graph is random or stratified sampling with no ordering to exploit. Our loss-based clustering approach presented in section 2.3 demonstrates that grouping by losses can provide an explanation for how skills are defined over data. An important direction for future work is to use such a clustering approach or other unsupervised algorithms in an end-to-end pipeline for skill discovery, skill graph learning, and data selection based on such skills.

B Additional Algorithmic Details

B.1 Derivation of SKILL-IT Update Rule

First, we provide the derivation of our update rule from online mirror descent using the proximal point view [18]. We restate our optimization problem from (3):

$$\begin{aligned} & \underset{p_1, \dots, p_T \in \Delta^{k-1}}{\text{minimize}} \quad \frac{1}{m} \sum_{j=1}^m L_{\text{eval},j}(f_T) \\ & \text{s.t.} \quad L_{\text{eval},j}(f_t) = L_{\text{eval},j}(f_{t-1})(1 - \alpha A_{:,j}^\top p_{t-1}) \quad \forall j \in [m], t = 1, \dots, T \\ & \quad f_t = \Phi(f_{t-1}, p_{t-1}) \quad \forall t = 1, \dots, T \end{aligned} \tag{5}$$

Let $\bar{L}_t(p) = \frac{1}{m} \sum_{i=j}^m L_{\text{eval},j}(f_{t+1}) = \frac{1}{m} \sum_{i=j}^m L_{\text{eval},j}(\Phi(f_t, p))$; that is, p is the mixture we must choose at time t and $\bar{L}_t$ is the average loss per skill of the model after it is trained on p at round t . A greedy approximation of (5) is $\underset{p \in \Delta^{k-1}}{\text{minimize}} \bar{L}_t(p)$, given the model and mixtures at previous rounds. A linear approximation of $\bar{L}_t(p)$ is

$$\bar{L}_t(p) \approx \bar{L}_t(p_{t-1}) + \langle \nabla \bar{L}_{t-1}(p_{t-1}), p - p_{t-1} \rangle \tag{6}$$

Then, the problem of minimizing $\bar{L}_t(p)$ can be approximated as

$$\underset{p \in \Delta^{k-1}}{\text{argmin}} \langle \eta \nabla \bar{L}_{t-1}(p_{t-1}), p \rangle \tag{7}$$

after we drop terms from (6) that do not depend on p . Note that the η is a constant and does not impact the solution. The optimal solution to this problem is selecting the p that has the most weight on the slice with the largest gradient (e.g., a follow-the-leader sort of algorithm). To improve stability and prevent overfitting, we introduce regularization via a Bregman divergence $D_h(p||p_{t-1}) = h(p) - h(p_{t-1}) - \langle \nabla h(p_{t-1}), p - p_{t-1} \rangle$. After dropping terms that do not contain p , our problem is now

$$\underset{p \in \Delta^{k-1}}{\text{argmin}} \langle \eta \nabla \bar{L}_{t-1}(p_{t-1}), p \rangle + h(p) - \langle \nabla h(p_{t-1}), p \rangle \tag{8}$$

Taking the gradient and setting it equal to 0 gives us

$$\eta \nabla \bar{L}_{t-1}(p_{t-1}) + \nabla h(p) - \nabla h(p_{t-1}) = 0 \tag{9}$$

Algorithm 2: LEARNGRAPH (Brute-Force)

```
1: Input: Ordered skill set  $\mathcal{S} = \{s_1, \dots, s_k\}$ . Number of training steps  $H$ , base model  $f$ .
2: for  $j \in [k]$  do
3:   Train  $f$  on samples from  $\mathcal{X}_{s_j}$  for  $H$  steps and denote  $f_{H,j}$  to be the model after training.
4:   Observe change in loss,  $\delta_j^j = L_{\text{eval},j}(f) - L_{\text{eval},j}(f_{H,j})$ .
5: end for
6: for  $i, j \in [k]$  do
7:   Train  $f$  on samples from  $\mathcal{X}_{s_i} \cup \mathcal{X}_{s_j}$  for  $H$  steps and denote  $f_{H,i,j}$  to be the model after training.
8:   Observe change in loss,  $\delta_j^{i,j} = L_{\text{eval},j}(f) - L_{\text{eval},j}(f_{H,i,j})$ .
9:   if  $\delta_j^{i,j} > \delta_j^j$  then
10:    Draw edge  $s_i \rightarrow s_j$  and set  $A_{ij} > 0$ .
11:   end if
12: end for
13: Return Adjacency matrix  $A \in \mathbb{R}^{k \times k}$ 
```

Algorithm 3: LEARNGRAPH (Approximate)

```
1: Input: Ordered skill sets  $\mathcal{S}_{\text{train}}$  and  $\mathcal{S}_{\text{eval}}$ . Number of training steps  $H$ , base model  $f$ .
2: for  $i \in [k]$  do
3:   Train  $f$  on samples from  $\mathcal{X}_{s_{\text{train},i}}$  for  $H$  steps and denote  $f_{H,i}$  to be the model after training.
4:   for  $j \in [m]$  do
5:     Observe change in loss,  $\delta_j^i = L_{\text{eval},j}(f) - L_{\text{eval},j}(f_{H,i})$ .
6:     If  $\delta_j^i > 0$ , draw edge  $s_{\text{train},i} \rightarrow s_{\text{eval},j}$  and set  $A_{ij} > 0$ .
7:   end for
8: end for
9: Return Bipartite adjacency submatrix  $A \in \mathbb{R}^{k \times m}$ 
```

Similar to in standard multiplicative weights, we set $h(p) = \sum_i p_i \ln p_i$ and $\nabla h(p) = [\ln p_i + 1]_i$. Then,

$$\begin{aligned} \ln p^i &= \ln p_{t-1}^i - \eta \nabla_i L_{t-1}(p_{t-1}) \\ \Rightarrow p_{t+1}^i &= p_t^i \exp(-\eta \nabla_i \bar{L}_t(p_t)) \end{aligned} \quad (10)$$

where ∇_i is the i th element of the gradient. Now we wish to compute $\nabla_i \bar{L}_t(p_t) = \frac{1}{m} \sum_{j=1}^m \nabla_i [L_{\text{eval},j}(f_{t+1})] = \frac{1}{m} \sum_{j=1}^m \nabla_i [L_{\text{eval},j}(\Phi(f_t, p_t))]$. Recall the dynamics model for L_{eval} :

$$L_{\text{eval},j}(f_{t+1}) = L_{\text{eval},j}(f_t)(1 - A_{:,j}^\top p_t), \quad (11)$$

The gradient of this model with respect to each training skill s_i is

$$\begin{aligned} \nabla_i L_{\text{eval},j}(f_{t+1}) &= -A_{ij} L_{\text{eval},j}(f_t) \\ \Rightarrow \nabla_i \bar{L}_t(p_t) &= \frac{1}{m} \sum_{j=1}^m -A_{ij} L_{\text{eval},j}(f_t) \end{aligned} \quad (12)$$

Plugging this back into (10),

$$p_{t+1}^i = p_t^i \exp\left(\eta \sum_{j=1}^m A_{ij} L_{\text{eval},j}(f_t)\right), \quad (13)$$

where we can absorb the $\frac{1}{m}$ into η .

B.2 Graph Learning Method

We provide algorithms for learning the graph over an ordered skill set. In Algorithm 2, we discuss the brute-force approach for learning the adjacency matrix. This approach only works when $\mathcal{S}_{\text{eval}} \subseteq \mathcal{S}_{\text{train}}$ (e.g. pre-training and fine-tuning

cases), so we denote $\mathcal{S} = \mathcal{S}_{\text{train}}$ in the algorithm box. In Algorithm 3, we discuss the linear approach for learning the adjacency matrix. This approach works even in the out-of-domain case when $\mathcal{S}_{\text{eval}}$ and $\mathcal{S}_{\text{train}}$ are disjoint.

In both approaches, the exact value of A_{ij} can vary, but we can typically set it proportional to $\delta_j^{i,j} - \delta_j^j$, the difference between the changes in loss, in the brute-force case or δ_j^i , the change in loss itself, in the approximate case. The exact constructions and methods for learning each A in our experiments are in Appendix C.2.

C Additional Experimental Details

C.1 Datasets

We present details about each dataset used, including information on the skills and the validation dataset. A summary is presented in Table 2.

Dataset	Skill	# skills	Validation data
Alpaca	Instruction type	38	50 samples per skill
Pile of Law	Legal data source	31	645 samples per skill
LEGO	Reasoning chain depth	5	100 samples per skill
Addition	Digit	3	100 samples per skill
NI (pre-training)	Task category	23	50 samples per task
NI (Spanish QG)	Task category $\times$ language	4	100 samples per task
NI (stance detection)	Task category	2	50 samples per task
NI (out-of-domain)	Task category	59, 12	400 samples per task
RedPajama	Data source	7	LM eval harness

Table 2: We list each dataset used as well as its corresponding skill. We include the number of skills in the training dataset, as well as details on how the validation dataset is constructed.

- Alpaca dataset [56]: the Alpaca dataset consists of 52K instruction examples that were generated from text-davinci-003. We applied the Berkeley Neural Parser [26, 27] to each instruction, keeping 40777 samples it was able to parse successfully. If the sample began with a question, we annotated it with the skill “question”, and otherwise we annotated it with the verb identified from the parser. We grouped the data into a total of 38 skills, such as “list”, “edit”, “calculate”, “describe” and “identify”.
- Pile of Law [21]: the Pile of Law dataset consists of various sources of legal and administrative data, ranging from tax rulings to the world’s constitutions. We evaluate on a subset of the Pile of Law validation dataset consisting of 13883 samples, where we selected $\max(645, \text{source size})$ samples per source. We truncated each sample to be no more than 100K characters.
- LEGO [72]: for the LEGO synthetic, we set $k = 5$ and sample 192000 points across the skills. Our validation dataset consisted of 100 samples per skill.
- Addition: for the 3-digit addition synthetic, we set $k = 3$ and sample 192000 points across the skills. We use a validation dataset of 100 samples per skill.
- Natural Instructions [40, 63]: the Natural Instructions dataset is a large collection of tasks and their definitions in natural language. For the pre-training setting, we used a set of 23 task categories that had the largest degree (in-degree + out-degree) in the learned skills graph, for a total of 1,232,437 samples and 425 tasks to select from. We evaluated on 50 samples per task.

For the fine-tuning setting with Spanish question generation, we select data over 4 skills (Spanish question generation, Spanish question answering, English question generation, English question answering) for a total of 513210 samples and 212 tasks to select from. We evaluated on 100 samples per task.

For the fine-tuning setting with stance detection, we select data over 2 skills (stance detection, text matching) for a total of 50990 samples and 19 tasks to select from. We evaluated on 50 samples per task.

For the out-of-domain setting, we select data over all 59 task categories for a total of 2,417,867 samples and 753 tasks to select from. The test split consisted of 12 task categories and 119 tasks, and we evaluated on $\min(400, \text{task size})$ samples per task.

- RedPajama [57]: the RedPajama dataset is a 1-trillion token dataset that aims to reproduce the LLaMA [59] training dataset. We select over the 7 data sources and evaluate using the LM evaluation harness [14].